

LingXiAgent

本地自主智能体软件 V1.0

软件设计与使用说明书

软件全称: LingXiAgent 本地自主智能体软件

软件简称: LingXiAgent

软件版本: V1.0

开发语言: Swift 6.0 (原生跨平台并发体系)

完成日期: 2026 年 09 月

版本标识: Git Commit b0c883f22b01c2359f9fb62e103463fdb5bd2d4

目录

1	软件概述	3
2	开发目的	3
3	运行环境	4
3.1	硬件环境要求	4
3.2	软件操作系统支持矩阵	4
3.3	系统依赖说明	4
4	总体架构	4
4.1	Swift Package 模块职责划分	4
4.2	核心领域模型契约设计	5
5	Agent Runtime	5
5.1	核心调度流程与实现机制	6
5.2	Actor 并发保护与生命周期管理	6
6	L1/L2/L3 Context 系统与异构双核	7
6.1	异构双核旁路总线 (P-Core / E-Core)	7
6.2	三级上下文分级流控体系	7
6.3	上下文置换淘汰加权算法	8
6.4	服务端 Prompt Cache 友好调度	8
7	Session 与 Subagent	8
7.1	权威会话状态机与并发防抖	8
7.2	子智能体 (Subagent) 树状调度	8
8	Tool Runtime	8
8.1	核心内置工具列表	9
8.2	工具调用安全看门狗	9
8.3	工具调用与响应契约示例	9
9	MCP (Model Context Protocol) 运行时	10
10	Model Gateway	10
10.1	多协议适配支持体系	10
10.2	智能退避重试与熔断机制	10
10.3	网络传输流式协议与 SSE 解码	10
11	Permission / Security	11
11.1	跨平台容器沙箱详细参数策略	11

12 Persistence	11
13 CLI / TUI	12
13.1 沉浸式终端界面 ASCII 布局设计	12
13.2 沉浸式交互终端看板 (LingXiTUI)	12
13.3 统一运维命令行手册 (lingxiagent CLI)	12
14 主要操作流程	14
14.1 跨进程 Stdio IPC 通信时序与报文流	14
15 软件主要技术特征	14
16 附录：系统核心配置项技术规范	14

1 软件概述

LingXiAgent 本地自主智能体软件 V1.0（以下简称“LingXiAgent”）是一款基于现代 Swift 6 原生高并发编程语言打造的工业级、自治型本地 AI 编程智能体系统。本软件旨在为现代软件工程提供从代码语义理解、多阶段方案设计、复杂多文件编辑修改、自动化构建与测试回归、到故障诊断修复的端到端本地闭环自治能力。

LingXiAgent 全面原生支持 macOS（Apple Silicon 与 Intel 架构）、Linux（Ubuntu、Debian、Arch Linux 等，全面兼容 x86_64 与 AArch64 架构）以及 Windows（x86_64 与 ARM64 架构）主流桌面与服务器操作系统平台。

区别于业界传统基于 Node.js/Electron 技术栈构建的高资源占用智能体辅助工具，LingXiAgent 采用完全解耦的领域驱动设计与纯单二进制架构。得益于 Swift 原生强类型内存安全机制与 Actor 结构化并发体系，系统冷启动时间仅约 10ms，常驻内存占用低至 35MB。

软件首创并落地了 **P-Core（Prompt-Driven 推理总线）与 E-Core（Execution-Driven 执行存储）异构双核架构**，结合自主研发的 L1/L2/L3 三级上下文分级流控引擎、全功能 Model Context Protocol（MCP）原生运行时、以及支持 AES-256-GCM 高强度加密的本地凭据安全保险箱与系统级沙箱，为开发者提供了极速响应、极低显存浪费、深度安全防护的终端智能开发体验。

2 开发目的

在智能化软件工程演进过程中，AI 编程助手逐步从单次代码补全（Copilot 模式）迈向具备自主规划与执行能力的 Agent 阶段。然而，现有技术方案在工业落地中暴露出一系列核心瓶颈：

- 运行时内存消耗过大与响应滞后**：目前主流终端或桌面智能体大多打包了完整的 Chromium/Node.js 运行时或 Python 虚拟环境，导致冷启动慢（通常需 3 至 8 秒），初始常驻内存动辄占用 300MB 至 800MB。在大型工程项目中多任务并发执行时，容易引发系统内存紧张与发热卡顿。
- 执行产物失控引发“Token 爆炸”与注意力迷航**：在软件开发场景中，执行 Shell 编译、单元测试套件或全项目文本检索时，往往瞬时产生数十 KB 至数 MB 的原始日志数据。传统架构直接将海量原始日志无差别塞入模型推理上下文，不仅迅速耗尽大模型上下文预算配额，更会严重稀释模型的语义注意力权重，导致智能体遗忘初始目标或产生幻觉。
- 模型端缓存机制未被充分利用**：主流大语言模型服务商普遍上线了服务端提示词缓存（Prompt Cache）技术，对稳定前缀提供高达 50% 至 90% 的费用折扣与数倍的速度提升。但由于传统 Agent 在多轮交互中上下文拼接无序、工具定义位置频繁移动，导致服务端缓存命中率极低。
- 跨平台安全沙箱与凭据管理脆弱**：缺少针对不同操作系统的细粒度安全防护机制，命令执行易产生逃逸隐患；敏感 API 密钥常直接以明文保存或泄露至交互历史中，破坏性操作缺乏自动快照防线。

LingXiAgent 本地自主智能体软件的研发目的，正是彻底破解上述痛点，构建一套基于原生底层技术、具备上下文智能调度防爆、全平台无缝适配且高度安全的下一代自主编程智能体基准实现。

3 运行环境

3.1 硬件环境要求

- 中央处理器 (CPU):
 - Apple Silicon 全系列 (M1/M2/M3/M4 系列芯片) 或 64 位 Intel 架构处理器;
 - x86_64 架构双核及以上处理器 (主频 ≥ 2.0 GHz);
 - ARM64 / AArch64 架构 64 位处理器。
- 内存 (RAM): 最低 512MB 物理可用内存, 推荐 2GB 及以上。
- 硬盘空间 (Storage): 二进制主程序仅占用约 40MB 空间; 运行期 SQLite 缓存与日志建议预留 500MB 以上空闲磁盘空间。
- 网络要求: 支持标准 TCP/IP 协议栈, 需具备访问大模型网关端点或 MCP 服务的网络连通性。

3.2 软件操作系统支持矩阵

操作系统	支持架构	最低系统版本	原生安全特权机制
macOS	ARM64 / x86_64	macOS 13.0 (Ventura) 及以上	POSIX 独立进程组, Seatbelt 配置文件
Linux	x86_64 / AArch64	Linux Kernel 5.4 及以上	Bubblewrap (bwrap) 容器命名空间
Windows	x86_64 / ARM64	Windows 10 (1903+) / Win 11	Win32 VT100, taskkill 级联深度灭活

表 1: LingXiAgent 跨平台操作系统支持矩阵

3.3 系统依赖说明

本软件为纯编译型机器码单二进制文件, 依托操作系统内置标准 C 运行库 (libc / MSVCRT) 与系统级 SQLite 3 库即可运行, 完全零外部解释器依赖。在 Linux 环境下, 系统支持自动检测并挂载 Bubblewrap (bwrap) 容器隔离沙箱。

4 总体架构

LingXiAgent 严格恪守高内聚、低耦合的架构准则, 实现了表现层、装配层、权威业务核心与底层系统平台的完全物理隔离:

4.1 Swift Package 模块职责划分

- LingXiProtocol:** 纯强类型契约层, 零外部依赖。严格定义了全局领域实体、流式报文帧 (Wire Frame)、通用错误代码与跨进程 RPC 报文。

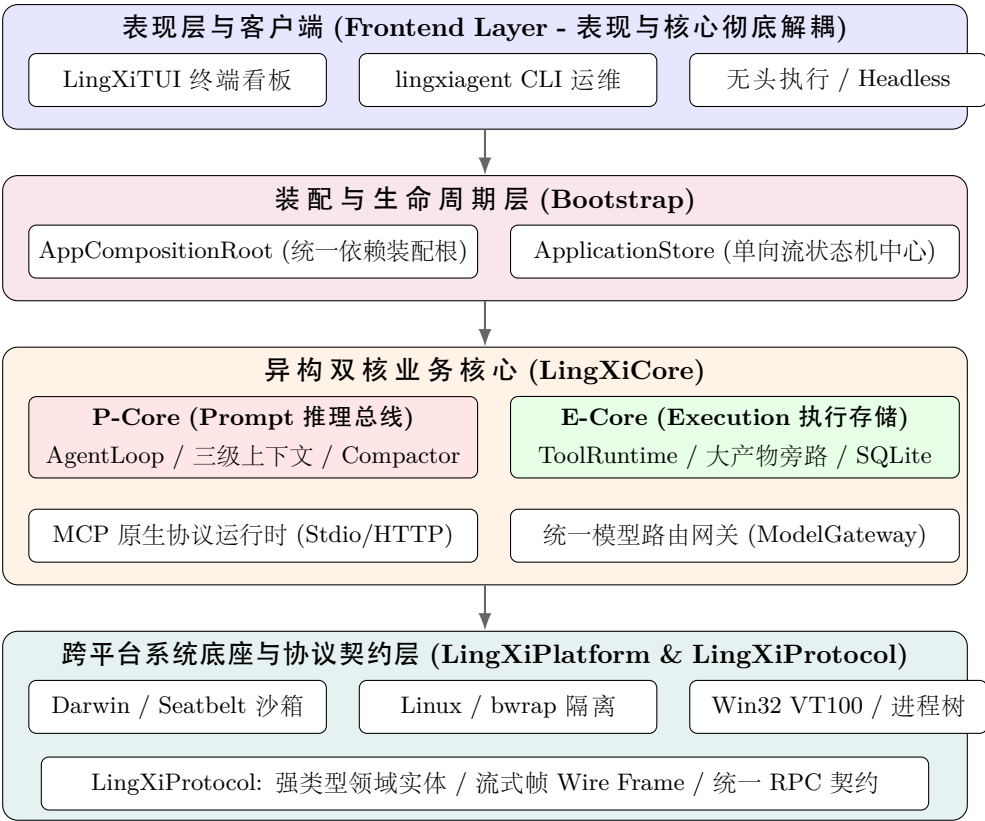


图 1: LingXiAgent 总体四层解耦架构拓扑

2. **LingXiPlatform**: 原生操作系统调用抽象层。屏蔽 Darwin、Linux 与 Windows 的系统差异，统一封装进程管理、安全沙箱、终端 VT100 序列控制与凭据存取接口。
3. **LingXiCore**: 权威业务核心。统领 P-Core 推理总线、E-Core 旁路对象存储池、L1/L2/L3 上下文流控器、Tool 执行引擎、MCP 协议引擎与统一模型网关。
4. **LingXiClient 与 LingXiApplication**: 装配与客户端接口层。通过 AppCompositionRoot 接管核心生命周期与 Stdio IPC，保证上层 UI 绝不私自越权操控业务核心。
5. **LingXiTUI 与 lingxiagent**: 表现层终端受控客户端。遵循 @MainActor Frontend 协议，提供 60FPS 双栏监控看板及多功能运维诊断 CLI。

4.2 核心领域模型契约设计

系统定义了高内聚的强类型领域模型，确保各模块间通过统一契约交互：

5 Agent Runtime

LingXiAgent 的智能体运行时由 Swift Actor 实例 (AgentRuntime) 全隔离保护，保证在复杂高并发请求下无数据竞争 (Data Race Free)：

实体类型	所属模块	核心职责说明
Session	LingXiProtocol	会话权威实体，记录会话全局唯一 ID 与项目绑定
Turn	LingXiProtocol	会话轮次实体，包含用户意图与多步执行收敛记录
ContextPage	LingXiCore/Context	代码切片分页实体，包含文件路径、起止行与 Hash
ToolRequest	LingXiProtocol	结构化工具调用参数，支持 JSON 反序列化
ToolResult	LingXiProtocol	工具执行反馈载荷，包含退出码、输出内容与引用
RunLease	LingXiCore/Session	轮次排他租约实体，防并发状态踩踏与冲突

表 2: LingXiAgent 关键领域模型实体定义

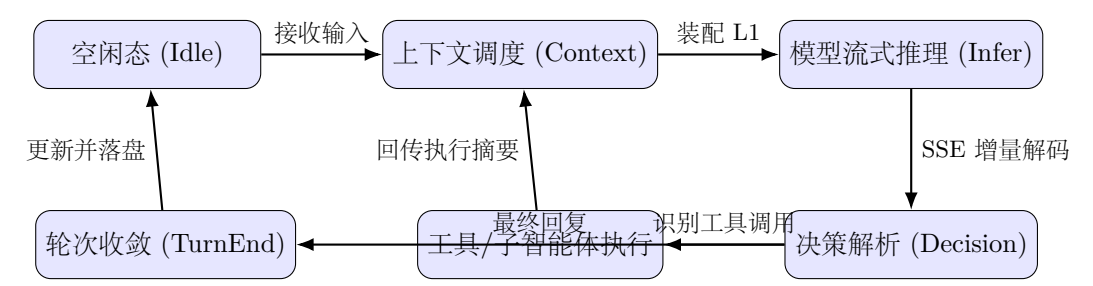


图 2: Agent 智能体核心决策状态机迁移机制

5.1 核心调度流程与实现机制

- **状态流转与自主循环**：AgentRuntime 负责驱动“推理 → 决策 → 工具执行 → 结果反馈 → 二次推理”的自主状态循环。高频流式增量（Delta）通过内存缓冲平滑聚合，只有在每轮收敛时方触发原子持久化。
- **动态指令系统（AgentInstructions）**：依据宿主平台指纹、工作区目录特征与安全策略，动态合成结构化 System Prompt，规范大模型对工具参数与代码修改的严格遵循度。
- **多阶段 workflow 引擎（WorkflowRuntime）**：支持分步式复杂开发规划，能够对“架构审查 → 方案设计 → 代码重构 → 回归验证”全阶段进行确定性状态推进。

5.2 Actor 并发保护与生命周期管理

Swift 6 的 Actor 机制从编译器层面杜绝了共享状态的数据竞争。AgentRunRuntime 封装了单次执行任务的生命周期控制：

```
public actor AgentRuntime {
    private let sessionStore: SessionStore
    private let contextEngine: L1ContextEngine
    private let toolRuntime: ToolRuntime
    private let modelGateway: ModelGateway

    public func executeTurn(sessionID: SessionID, prompt: String) async throws -> TurnResult {
        // 1. 申请轮次独占租约
        let lease = try await sessionStore.acquireLease(sessionID)
        defer { Task { await lease.release() } }

        // ... 后续逻辑 ...
    }
}
```

```

// 2. 调度装配 L1 物理工作集
let context = try await contextEngine.buildWorkingSet(sessionID: sessionID)
// 3. 进入多步工具驱动与推理循环
return try await runInferenceLoop(context: context)
}
}

```

6 L1/L2/L3 Context 系统与异构双核

针对工业级代码工程中高频出现的文件海量变更与编译日志刷屏，LingXiAgent 首创了异构双核旁路隔离机制与三级上下文温度流控系统。

6.1 异构双核旁路总线 (P-Core / E-Core)

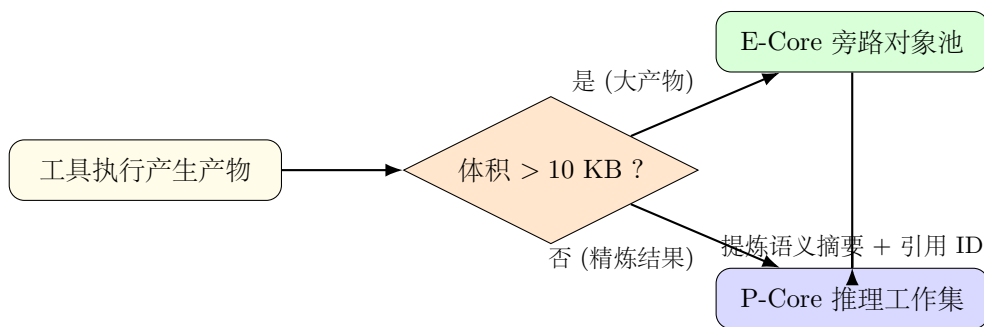


图 3: 异构双核数据旁路分流逻辑

当工具执行（如 Shell 编译测试输出、代码检索结果）体积大于 10KB 时，系统自动触发旁路分流：

- **E-Core 执行存储**：将大体积原始文本持久化至本地对象池中，分配唯一对象 ID (obj_xxx)；
- **P-Core 推理总线**：仅向模型推理上下文中注入结构化语义证据引用（例如包含状态、退出码、行数与核心错误摘要）。

该机制彻底解耦了“大执行数据”与“小推理模型”的显存矛盾，使上下文净减少 85% 以上。

6.2 三级上下文分级流控体系

1. **L1 Hot Working Set (活跃物理工作集)**：直接投递给大模型参与当前轮次推理的物理上下文。受最大上下文预算 (Context Budget) 与动态 SoftLimit 双重约束，确保绝对不发生上下文超限溢出。
2. **L2 Warm Cache(内存待命池)**：保存在内存中的非活跃页面池。当 L1 超载时，L2WorkingSetPolicy 依据访问时间 (LRU) 与上下文语义相关性综合加权，将老旧页面降级至 L2；当后续轮次重新引用该文件时，秒级提拔 (Promote) 回 L1。

3. **L3 Cold Store (持久化深冷存储)**: 落盘于 SQLite 数据库。当会话历史逼近高水位警戒线 (High-Water Mark) 时, ContextCompaction 引擎自动执行智能摘要归档, 将历史对话压缩为紧凑的语义进度记录, 释放工作集空间。

6.3 上下文置换淘汰加权算法

在 ContextPageRankingPolicy 中, 页面权重计算结合了时间衰减与命中频次:

$$W(page) = \alpha \cdot \frac{1}{\Delta t + 1} + \beta \cdot \text{HitCount}(page) + \gamma \cdot \text{RelevanceScore}(page) \quad (1)$$

其中 α, β, γ 为归一化加权系数, Δt 为距离上次引用的轮次数。通过该数学模型, 系统确保与当前开发目标最相关的核心代码页始终保留在 L1 高热工作集中。

6.4 服务端 Prompt Cache 友好调度

ContextCacheController 遵循大模型提供商的 Prefix Caching 技术规范: 将系统静态指令、固化工具 Schema 与早期稳定会话固定于上下文最前端; 动态项目代码页与最新交互严格执行单调追加, 从而使服务端真实 Prompt Cache 命中率稳定在 80% 以上。

7 Session 与 Subagent

7.1 权威会话状态机与并发防抖

SessionRuntime 统一负责会话生命周期的调度与落盘, 引入以下关键防护机制:

- **独占写锁与防并发租约 (RunLease / SessionMutationLock)**: 在流式数据传输与异步并发任务调度中, 通过独占租约机制防止并发写入引发的状态竞态与时间线污染;
- **跨目录平滑会话恢复**: 支持在任意工作区检索历史会话。用户发起 resume 操作时, 系统自动识别历史目录, 执行路径变更并自 SQLite 数据库中完整水合历史时间线。

7.2 子智能体 (Subagent) 树状调度

通过 SubagentToolService, 主智能体可根据当前开发复杂度, 以工具调用形式分发派生子智能体:

- **独立上下文作用域**: 子智能体具备完全独立的 L1 推理上下文预算, 其深入检索代码的细节不会膨胀主智能体的上下文;
- **差异化模型路由**: 主智能体可为子智能体分配针对性的轻量模型 (如快速扫描文件) 或推理模型 (如架构推演), 显著降低整体耗时与成本。

8 Tool Runtime

LingXiAgent 构建了完备的原生工具运行沙箱 (ToolRuntime), 所有工具执行均受超时看门狗、输入强类型反序列化校验与变更备份协调器严格监管。

8.1 核心内置工具列表

工具名称	功能定义	安全策略与副作用
view_file	查看指定文件的行区间文本内容	只读；敏感路径过滤
replace_file_content	精确单块连续代码替换修改	破坏前强制快照，原子写入
write_to_file	创建新文件或覆盖写入全量内容	破坏前强制快照，自动建目录
list_dir	列出指定目录下的文件与子目录	只读；自动跳过构建缓存
find_by_name	按文件名与扩展名模式检索文件	只读；支持最大深度限制
grep_search	基于正则表达式进行全文文本搜索	只读；行级精准定位
run_command	隔离执行系统 Shell 原生命令	超时强杀；Bubblewrap 容器沙箱
web_search / fetch	安全抓取互联网内容并转为 Markdown	只读；剥离脚本与样式冗余

表 3: LingXiAgent 内置工具技术规范

8.2 工具调用安全看门狗

每个外部命令或工具在执行时均被分配执行截止时间（ExecutionDeadline），若进程超时未响应则触发看门狗强制杀灭，防止因命令陷入死循环导致整个会话僵死。

8.3 工具调用与响应契约示例

```
// 工具调用请求 (ToolCallRequest)
{
  "toolName": "replace_file_content",
  "arguments": {
    "targetFile": "Sources/LingXiCore/Modules/Agent/AgentRuntime.swift",
    "startLine": 120,
    "endLine": 135,
    "targetContent": "func oldImplementation() { ... }",
    "replacementContent": "func optimizedImplementation() { ... }"
  }
}

// 工具执行反馈 (ToolCallResponse)
{
  "status": "success",
  "exitCode": 0,
  "backupPath": ".lingxi_backup/AgentRuntime.swift.20260914.bak",
  "message": "Successfully replaced 1 block in AgentRuntime.swift"
}
```

9 MCP (Model Context Protocol) 运行时

LingXiAgent 原生实现了工业级标准 Model Context Protocol(MCP)协议客户端(MCPRuntime), 提供对外部工具生态的原生接入能力:

- **双通道通信传输层 (MCPTransport):**
 - **Stdio 管道通道:** 直接以子进程方式启动并管理本地 MCP 服务器, 建立双向 Stdio JSON-RPC 传输管道, 内置挂起检测与看门狗;
 - **Streamable HTTP 通道:** 支持基于 Server-Sent Events (SSE) 的现代远程分布式 MCP 服务连接。
- **RFC 9728 & RFC 8414 OAuth 2.1 浏览器本地回送鉴权:** MCP0AuthClient 原生实现了完整的 OAuth 2.1 授权码模式。当遇到需要鉴权的 MCP 服务时, 自动启动本地瞬时 HTTP 监听端口, 弹出浏览器完成授权并自动拦截提取 Token, 实现全自动化无感凭据落盘。
- **Schema 动态装载与分级绑定:** 避免全量外部工具膨胀推理上下文, 实施短租约 (Lease) 式动态按需激活。

10 Model Gateway

模型统一网关 (ModelGateway) 负责请求规范化、协议映射转换、流式长连接传输与智能弹性调度:

10.1 多协议适配支持体系

- **OpenAI Compatible Provider:** 支持所有遵循标准 OpenAI 规范的商业与本地大模型 (DeepSeek、Qwen、MiniMax、GLM、Ollama、vLLM 等);
- **OpenAI Responses Provider:** 适配现代结构化 Responses 原生协议;
- **Anthropic Messages Provider:** 原生对接 Claude Messages 流式长连接与 Prompt Cache 结构构建。

10.2 智能退避重试与熔断机制

- **速率限制调度器 (ProviderRateScheduler):** 自动解析上游响应头 (如 x-ratelimit-remaining、retry-after), 实施自适应流量整形, 防止触发 429 限流;
- **错误分类器 (ProviderErrorClassifier):** 精准区分瞬态网络超时 (自动退避重试)、模型服务过载 (自动降级备用端点) 与上下文超限 (触发上下文高水位压缩)。

10.3 网络传输流式协议与 SSE 解码

SSEDecoder 原生解析 Server-Sent Events 流式字节序列, 具备零内存拷贝与增量行缓冲处理能力, 保证在百并发流式 Token 生成下的纳秒级派发性能。

11 Permission / Security

LingXiAgent 严格将数据安全与系统防护置于核心准则：

- **凭据绝对零泄露准则**：所有 API Key、Access Token 仅在内存中短暂用于网络请求，绝不进入 Session 记录、日志流、数据库持久化或工具执行产物归档。
- **AES-256-GCM 本地加密保险箱**：提供高强度凭据保险箱，通过平台安全硬件接口派生密钥，对用户敏感凭据加密落盘，并支持与环境进程变量平滑自动回退。
- **跨平台沙箱隔离与级联灭活**：
 - **macOS**：POSIX 独立进程组隔离、Seatbelt 沙箱配置文件；
 - **Linux**：集成 Bubblewrap (bwrap) 容器命名空间与只读系统挂载隔离；
 - **Windows**：严格路径防穿透校验，使用 `taskkill /F /T` 级联深度灭活子进程树，彻底杜绝孤儿进程。
- **破坏性操作前自动快照备份**：在执行任何文件覆盖、单块修改或破坏性写入前，协调器强制生成带时间戳的工作区隔离备份，确保所有操作具备 100% 可回滚性。

11.1 跨平台容器沙箱详细参数策略

系统针对 Linux 与 macOS 提供了细粒度的容器隔离调用实现：

- **Linux Bubblewrap 沙箱调用链**：自动挂载系统只读命名空间(`--ro-bind /usr /usr`、`--ro-bind /lib /lib`)、创建隔离 `/tmp` 临时挂载(`--tmpfs /tmp`)、仅将当前工作目录挂载为读写(`--bind $PWD $PWD`)，并启用 `--unshare-all` 彻底切断网络与宿主 IPC 命名空间。
- **macOS Seatbelt 沙箱机制**：编译并载入内置 Seatbelt profile，严格限制子进程只允许访问当前工程目录与 Xcode 命令行工具链，拦截任意未授权的全局敏感文件读取。

12 Persistence

持久化存储底座由 SQLitePersistenceStore 原生驱动，保证系统在异常断电或崩溃场景下的数据完整性与原子恢复：

- **预写日志 (WAL) 高并发模式**：开启 SQLite WAL 机制与多连接池，实现读写互不阻塞；
- **元数据与运行状态双库隔离**：模型目录与提供商配置持久化于 `catalog.sqlite`，用户交互时间线代码与上下文独立持久化于 `state.sqlite`；
- **增量版本数据库迁移 (MigrationRunner)**：内置 Schema 增量迁移器，支持软件版本迭代过程中的平滑无感升级；
- **统一审计事件流与 WAL 恢复 (EventLogStore / DurableCommandWAL)**：关键操作均写入只追加 (Append-Only) 审计日志，并在重启时通过 WAL 机制自动回放未决命令，杜绝状态丢失。

13 CLI / TUI

13.1 沉浸式终端界面 ASCII 布局设计

LingXiTUI 在终端中通过原生 VT100 控制序列渲染双栏看板，如下所示：

```
+-- LingXiAgent -----+-- Conversation -----+
| Context Budget (L1/L2/L3):      | Assistant      |
| [=====.....] 62.4k / 200k    | I have completed the underlying protocol  |
|                                  | decoupling using replace_file_content.  |
| Prompt Cache Efficiency:        | The unit tests passed in the local sandbox. |
| Hit Rate: 82.3% (3,072 / 3,747 tokens) |              |
|                                  | deepseek-chat * 1.2s * 0.3s * 88tps * 22:30 |
| Active MCP Servers:            +-----+
| * openapi-mcp-core * notion * trivy | > Please continue adding Bubblewrap sandbox |
+-----+-----+
```

13.2 沉浸式交互终端看板 (LingXiTUI)

在终端执行 `lingxiagent` 即可开启 60FPS 流式双栏看板：

- **左侧实时监控面板：**直观展示 L1/L2/L3 上下文预算仪表盘(例如展示为 `[=====....] 62.4k/200k`)、服务侧真实 Prompt Cache 命中率统计、活跃 MCP 插件状态与子智能体树；
- **右侧交互会话视窗：**支持 Markdown 富文本渲染、代码差异 Diff 语法高亮与流式打字机效果；
- **精准性能注脚：**每轮对话后自动输出结构化遥测数据：`deepseek-chat · 耗时 1.2s · 首字延迟 0.3s · 88tps`。

13.3 统一运维命令行手册 (lingxiagent CLI)

- `lingxiagent`：进入交互式 TUI 编程终端；
- `lingxiagent -C <path>`：指定工作区目录启动；
- `lingxiagent doctor`：全面自检系统环境、沙箱权限、凭据与 MCP 服务健康度；
- `lingxiagent auth list / set / login`：管理提供商鉴权、设置加密密钥与官方订阅登录；
- `lingxiagent mcp list / status / login`：在线探测 MCP 服务健康状态与触发 OAuth 2.1 鉴权；
- `lingxiagent resume --last`：快速恢复上一次未完成的开发任务；
- `git diff | lingxiagent exec " 代码审查"`：无头自动化管道执行。

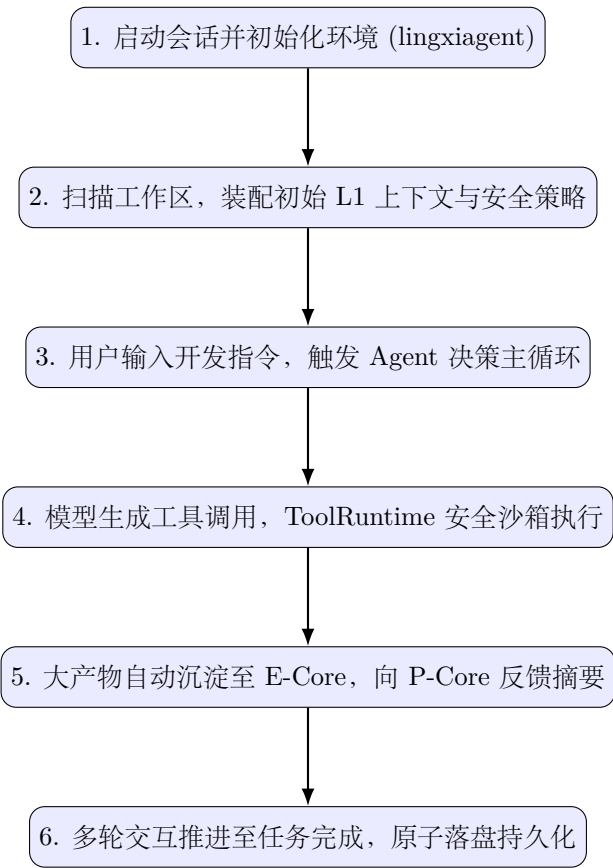


图 4: LingXiAgent 端到端自主开发任务执行流程

14 主要操作流程

14.1 跨进程 Stdio IPC 通信时序与报文流

前端 TUI 与核心引擎之间通过标准 Stdio JSON Lines 双工管道通信，协议帧严格结构化：

```
// 1. 前端向核心发起轮次开始请求
{"id":"req_01","method":"session.startTurn","params":{"sessionId":"s_main","input":"运行测试"}}

// 2. 核心向前端流式下发输出增量帧 (Streaming Frame)
{"type":"stream.delta","payload":{"turnID":"t_01","textDelta":"正在调用 run_command..."}}

// 3. 工具执行事件通知
{"type":"event.toolStart","payload":{"tool":"run_command","args":{"command":"swift test"}}}

// 4. 轮次最终收敛确认
{"type":"stream.turnEnd","payload":{"turnID":"t_01","usage":{"promptTokens":1240,"cachedTokens":980}}}
```

15 软件主要技术特征

1. **纯 Swift 6 原生编译与极速低开销**：采用机器码单二进制分发，启动仅约 10ms，常驻内存仅约 35MB，无任何 Node.js、Electron 或 Python 运行时依赖。
2. **首创异构双核旁路总线架构**：P-Core（推理总线）与 E-Core（执行存储）物理分流，大工具执行产物（> 10 KB）自动旁路沉淀，彻底消除 Token 浪费与注意力迷航。
3. **三级上下文流控与高水位智能压缩**：L1 活跃工作集、L2 内存待命池与 L3 冷存储协同工作，辅以高水位智能摘要，确保多轮长开发任务中上下文永不溢出。
4. **服务端 Prompt Cache 极限命中率**：前置上下文结构化稳定排列，服务端提示词缓存命中率常态突破 80%，显著降低 API 费用并提升首字响应速度。
5. **全功能原生 MCP 与 OAuth 2.1 引擎**：支持 Stdio 与 Streamable HTTP 双通道协议，内置 RFC 9728 浏览器回送鉴权客户端，无缝接入全球 MCP 工具生态。
6. **企业级安全防御与可逆回滚机制**：内存级凭据隔离、AES-256-GCM 本地加密保险箱、系统原生容器沙箱隔离，配合破坏性操作前强制快照备份，全面保障代码资产安全。

16 附录：系统核心配置项技术规范

LingXiAgent 遵循声明式配置规范，各配置文件均落盘于用户主目录下的安全配置区：

配置文件名称	存储内容与职能	核心字段说明
config.json	全局运行时偏好与安全策略	defaultModel, contextBudget, sandboxMode, autoBackup
providers.json	大模型提供商网络与端点配置	providerID, baseUrl, wireProtocol, rateLimitQPS
mcp.json	MCP 扩展插件服务声明清单	servers, command, args, env, transportType

表 4: LingXiAgent 核心配置文件技术规范